

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE:CSA5115 COURSE NAME: Cryptography and Network Security
SLOT:A

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A ***Coding Challenge***

Code of Conduct:

I Nissanki Sri Venkata Lakshmi kavya (192373036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.S.V.L.Kavya

Annexure A
Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <openssl/hmac.h>
#include <openssl/evp.h>

void generate_mac(const char *message, const char *key,
                 unsigned char *mac, unsigned int *mac_len) {

    HMAC(EVP_sha256(),
        key, strlen(key),
        (unsigned char *)message, strlen(message),
        mac, mac_len);
}

void print_mac(unsigned char *mac, unsigned int len) {
    for (unsigned int i = 0; i < len; i++) {
        printf("%02x", mac[i]);
    }
    printf("\n");
}

int main() {
    char packet[200];
    char key[100];
```

```

unsigned char sender_mac[EVP_MAX_MD_SIZE];
unsigned char receiver_mac[EVP_MAX_MD_SIZE];

unsigned int sender_len;
unsigned int receiver_len;

printf("Enter IP packet data: ");
fgets(packet, sizeof(packet), stdin);
packet[strcspn(packet, "\n")] = '\0';

printf("Enter secret key: ");
fgets(key, sizeof(key), stdin);
key[strcspn(key, "\n")] = '\0';

/* Sender: Generate AH authentication MAC */
generate_mac(packet, key, sender_mac, &sender_len);

printf("\n--- IPSec AH Simulation ---\n");
printf("Original IP Packet : %s\n", packet);

printf("Generated MAC    : ");
print_mac(sender_mac, sender_len);

/* Receiver: Recalculate MAC */
generate_mac(packet, key, receiver_mac, &receiver_len);

printf("Verified MAC      : ");
print_mac(receiver_mac, receiver_len);

/* Compare MACs */
if (sender_len == receiver_len &&
    memcmp(sender_mac, receiver_mac, sender_len) == 0) {

```

```

printf("\nAuthentication successful!\n");
printf("IP packet integrity verified.\n");

} else {

printf("\nAuthentication failed!\n");
printf("IP packet may have been modified.\n");
}

return 0;
}

```

OUTPUT:

The screenshot shows a Visual Studio Code editor with a C program named `practice1.c` open. The code defines a function `generate_mac` that calculates a MAC value based on a message and a key. The `main` function simulates an IPsec AH simulation, entering IP packet data and a secret key, and then prints the results of the MAC calculation and integrity verification.

The terminal output shows the following commands and results:

```

PS C:\Users\sruni\OneDrive\Desktop\practice programmes> gcc practice1.c -o practice1 -lcrypto
PS C:\Users\sruni\OneDrive\Desktop\practice programmes> gcc practice1.c -o practice1 -lcrypto
C:\mingw\bin/../lib/gcc/mingw32/6.3.0/../../../../mingw32/bin/ld.exe: cannot find -lcrypto
collect2.exe: error: ld returned 1 exit status
PS C:\Users\sruni\OneDrive\Desktop\practice programmes> gcc practice1.c -o practice1
PS C:\Users\sruni\OneDrive\Desktop\practice programmes> .\practice1.exe
Enter IP packet data: HELLO IPSEC
Enter secret key: secret123

--- IPsec AH Simulation ---
IP Packet : HELLO IPSEC
Generated MAC : 911
Verified MAC : 911

Authentication Successfull
IP Packet Integrity Verified.
PS C:\Users\sruni\OneDrive\Desktop\practice programmes>

```

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

void encrypt(char *data, const char *key)
{
    int keyLen = strlen(key);

    for (int i = 0; data[i] != '\0'; i++)
        data[i] = data[i] ^ key[i % keyLen];
}

void decrypt(char *data, const char *key)
{
    encrypt(data, key);
}

int main()
{
    char payload[200];
    char key[100];

    printf("Enter IP packet payload: ");
    fgets(payload, sizeof(payload), stdin);
    payload[strcspn(payload, "\n")] = '\0';

    printf("Enter secret key: ");
    fgets(key, sizeof(key), stdin);
```

```

key[strcspn(key, "\n")] = '\0';

printf("\n--- IPsec ESP Simulation ---\n");

encrypt(payload, key);

printf("Encrypted Payload: ");

for (int i = 0; payload[i] != '\0'; i++)

    printf("%02X ", (unsigned char)payload[i]);

printf("\n");

decrypt(payload, key);

printf("Decrypted Payload: %s\n", payload);

return 0;

}

```

OUTPUT:

The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** Lists files in the 'practice programmes' folder, including Experiment-17.c through Experiment-25.c, practice1.c, and practice2.c.
- EDITOR:** Displays the source code for practice2.c, which implements an IPsec ESP simulation. The code includes functions for encryption and decryption, and a main function that prompts the user for a payload and a secret key.
- TERMINAL:** Shows the execution of the program. The output is as follows:


```

PS C:\Users\sruini\OneDrive\Desktop\practice programmes> .\practice1.exe
--- IPsec AH Simulation ---
IP Packet      : HELLO IPSEC
Generated HMAC : 911
Verified HMAC  : 911

Authentication Successful!
IP Packet Integrity Verified.
PS C:\Users\sruini\OneDrive\Desktop\practice programmes> gcc practice2.c -o practice2
PS C:\Users\sruini\OneDrive\Desktop\practice programmes> .\practice2.exe
Enter IP packet payload: HELLO IPSEC
Enter secret key: KEY

--- IPsec ESP Simulation ---
Encrypted Payload: 03
Decrypted Payload: H
PS C:\Users\sruini\OneDrive\Desktop\practice programmes>

```

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases